

Southwest
Tech

Python Programming Foundations

Week 2 - Decision Logic and Repetition
Day 3: "Combining Logic and Repetition"

1

Chose And Repeat

This week, you learned two control-flow tools.
Decisions choose what happens.
Loops repeat what happens.

Today, those ideas work together:

- ▶ a condition can choose a branch
- ▶ a loop can repeat a process
- ▶ a condition can also help decide when repetition stops

The diagram shows a 'choose' box (with True/False paths) plus a 'repeat' box (with a loop arrow) equals a 'small program' box containing a code snippet: `if condition: do something else: do something else: do something`.

2

Monday Plus Tuesday

Monday: conditions choose branches.
Tuesday: loops repeat work.

Thursday's job is to keep both ideas explainable:

- ▶ What condition is checked?
- ▶ What repeats?
- ▶ What changes?
- ▶ What stops the process?

3

Week Recap

Monday: conditions choose
Tuesday: loops repeat

Monday flowchart: A diamond 'condition?' leads to 'True' or 'False' paths, which then lead to a 'stop' box.

Tuesday flowchart: A circle 'do something' leads to a 'stop' box.

Thursday: combine carefully

Monday Plus Tuesday

4

What Counts as Success Today

A combined program should still be traceable.

PATH: input or value -> condition -> branch or loop -> update -> output

If the path is hard to explain, reduce the program.

5

Program Path

A simple path a program follows.

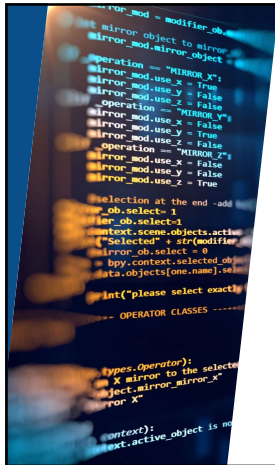
The flowchart shows a sequence of steps: 1. input or value (Get data or a starting value), 2. condition (Check a condition), 3. branch or loop (Choose a path or repeat), 4. update (Change a value or move to the next step), and 5. output (Show or print the result). A feedback loop goes from the output back to the condition.

Legend:
→ Highlighted path: one possible route
--- Loop back: repeat until condition is false

Programs follow paths. Conditions decide. Loops repeat. Updates move us forward.

Today's Success Pattern

6



What We Will Use Today

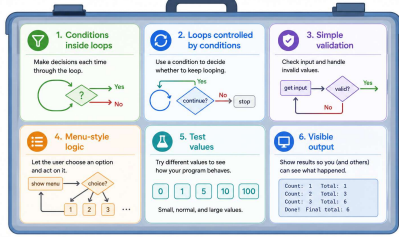
- ▶ conditions inside loops
- ▶ loops controlled by conditions
- ▶ simple validation
- ▶ small menu-style logic
- ▶ test values and visible output

The goal is controlled combination, not feature count.

7

Loop & Logic Toolbox

Simple tools for building strong programs



Today's Toolbox

8

What We Will "Skip" For Now

- ▶ large interactive applications
- ▶ complicated nested logic
- ▶ Avoid adding features until the main path is clear.

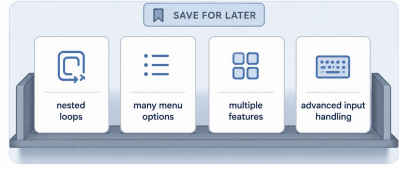
Today, one condition plus one repetition pattern is enough for success.



9

Parked For Later

These ideas are useful, but they are not today's focus.



Useful ideas, saved until the main path is clear.

"Parked"

10

Conditions Can Control Loops

A loop can use a condition to decide whether repetition should continue.

Example questions:

- ▶ Is the user done?
- ▶ Is the count still above zero?
- ▶ Is the input valid yet?
- ▶ Should the menu appear again?



11

Validation Is Logic Plus Repetition

Validation often means repeating until the value is acceptable.

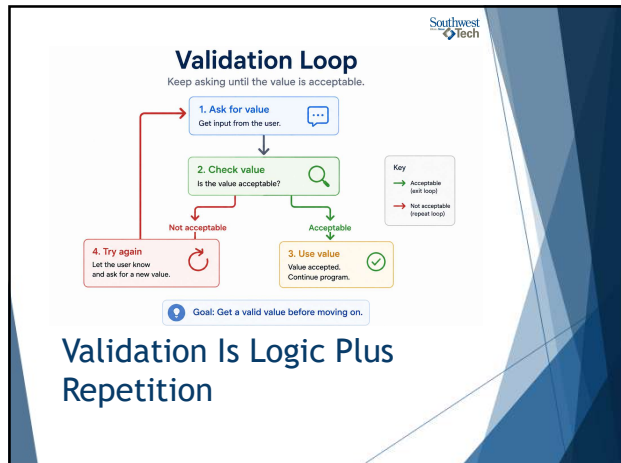
The pattern is:

- ▶ ask for a value
- ▶ check the value
- ▶ repeat if it is not acceptable
- ▶ continue when it is acceptable

This is logic and repetition working together.



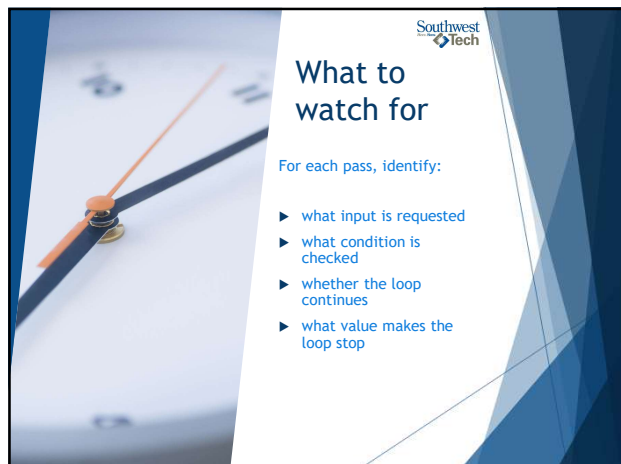
12



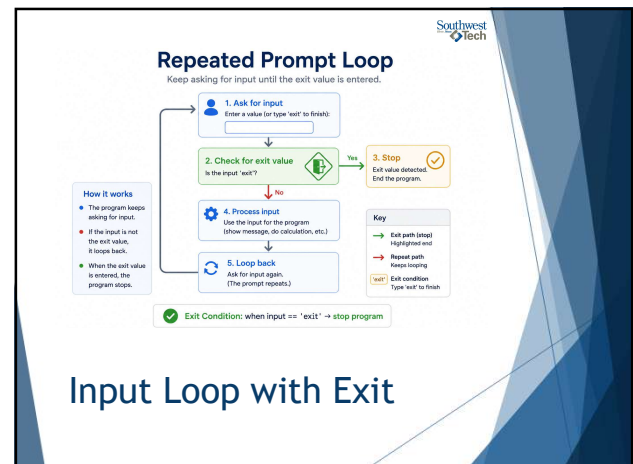
13



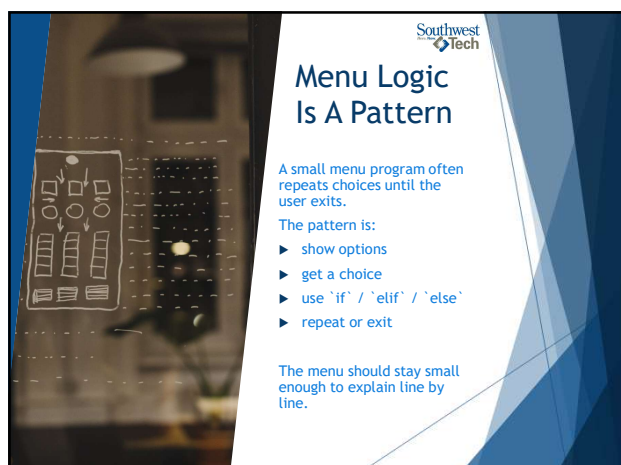
14



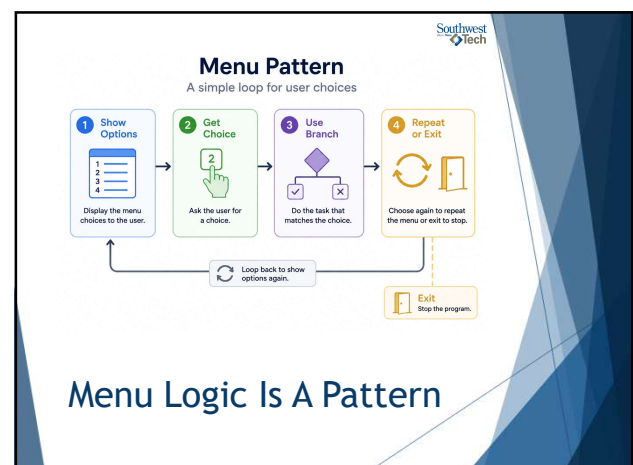
15



16



17



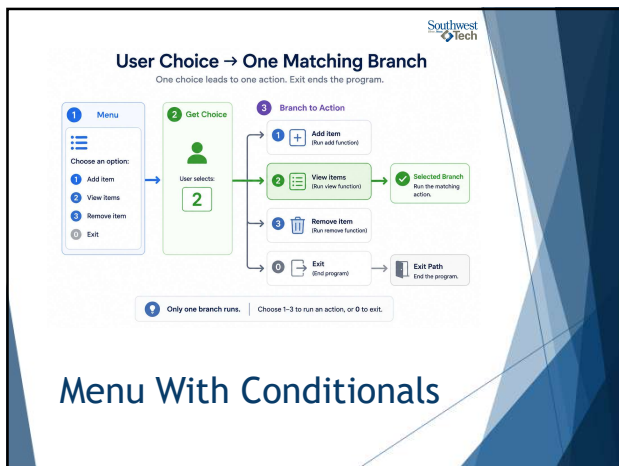
18



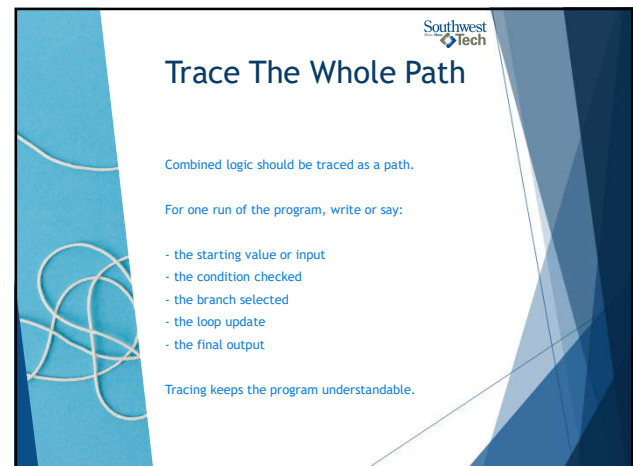
19



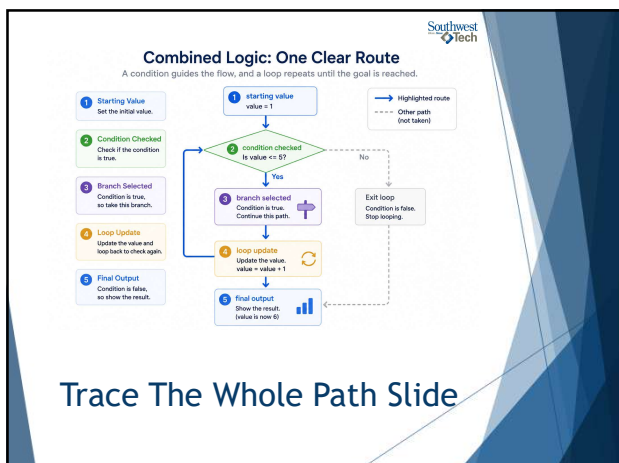
20



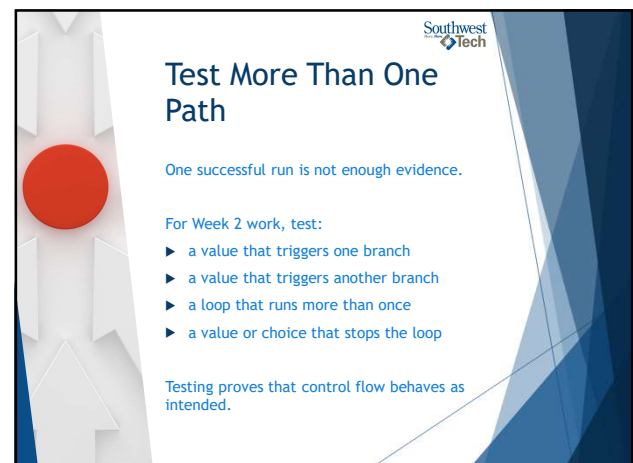
21



22



23



24

Quick Logic Checks
Run small tests to build confidence in your logic.

1 **Branch A**
Condition checked
A
B
Action A
Action B
Test that the program takes Branch A when the condition is true.
Output: Action A ran as expected. ✓

2 **Branch B**
Condition checked
A
B
Action A
Action B
Test that the program takes Branch B when the condition is false.
Output: Action B ran as expected. ✓

3 **Repeat More Than Once**
Loop body
Test that the loop runs multiple times before stopping.
Output: Loop ran 3 times as expected. ✓

4 **Stop Condition**
STOP
Test that the program stops when the stop condition is met.
Output: Stop condition triggered. ✓

✓ All tests passed. Your logic is working as expected.

Test More Than One Path

25

Common Failure: Too Much At Once

Too many features can break understanding.
If the program is hard to trace, reduce it.

Good Week 2 scope:

- ▶ one clear decision
- ▶ one clear loop
- ▶ visible output
- ▶ a short explanation

26

Focus First, Then Expand
Master one condition and one loop before adding more.

✓ **Small, Traceable Program (One Condition + One Loop)**

1. Get starting value
count = 1

2. Check condition
Is count <= 5?

Yes → 3. Do work
Print count

4. Update
count = count + 1

Output result Done.

Preferred Path
Easy to follow, easy to test, easy to fix.

✗ **Cluttered Mini-Application (Too Much, Too Soon)**

Harder Path
Hard to follow, hard to test, hard to maintain.

★ Start small. Build confidence. Add more only when the core is clear.

Common Failure: Too Much At Once

27

Common Failure: Hidden Stop Logic

A loop should not hide its stopping point.

Before submitting, ask:

- ▶ What makes the loop continue?
- ▶ What makes the loop stop?
- ▶ Can the stop condition actually happen?
- ▶ Can I explain it without guessing?

28

Loop Cycle with Clear Stop Condition
The loop repeats while the condition is true. It stops when the condition becomes false.

1. Start
Initialize values

2. Check Condition
Is the condition true?

No (false) → Stop Condition Spotlight

Yes (true) → 3. Do Work
Perform the loop tasks

4. Update
Make progress toward the stop condition

Stop Condition Spotlight
Can this condition actually become false?
Examples:
• counter <= 5
• items remaining > 0
• not done
• user choice != exit

No (false) → Exit Loop
Condition is false.
Continue the program.

✓ A good loop makes progress each time and will eventually stop. If the condition can never become false, the loop will run forever.

Common Failure: Hidden Stop Logic

29

Finish Assignment 2

For Assignment 2, check that your decision program has:

- ▶ one clear condition
- ▶ at least two possible outcomes
- ▶ output that matches the selected branch
- ▶ variable names that make the decision readable

Be ready to explain why each branch runs.

30



Southwest Tech

Finish Assignment 3

For Assignment 3, check that your loop program has:

- ▶ one repeated action
- ▶ a clear update or advance
- ▶ a clear stopping point
- ▶ output that proves repetition happened

Be ready to explain the first pass, one middle pass, and the stopping moment.

31



Southwest Tech

Evidence for Week 2

Week 2 submissions should show both working code and reasoning.

Useful evidence:

- ▶ working `.py` files
- ▶ visible output
- ▶ more than one test value or path
- ▶ explanation of the decision
- ▶ explanation of the loop stopping point

The explanation proves that the code is yours to understand.

32

Southwest Tech

Evidence of Core Logic

Two small programs, two ideas proven.

decision_logic.py
(Decision / Branch)

This program checks a value and chooses a branch.
If the value is 10 or more, it prints "High". Otherwise, it prints "Low".

Sample Output

Enter a value: 12
High

Enter a value: 7
Low

Explanation: Decision Logic

The program uses one condition to choose between two actions. When the value is 12, it takes the High branch. When the value is 7, it takes the Low branch.

count_until_exit.py
(Loop / Repeated Prompt)

This program keeps asking for numbers and adds them up.
It stops when the user enters 0.

Sample Output

Enter a number (0 to stop): 4
Enter a number (0 to stop): 6
Enter a number (0 to stop): 3
Enter a number (0 to stop): 8
Total = 13

Explanation: Loop Stopping Logic

The loop repeats until the user enters 0. When 0 is entered, the condition becomes false and the program stops, showing the final total.

Code + Output + Explanation = Understanding Small programs. Clear ideas. Proven logic.

Evidence for the week

33



Southwest Tech

Closing Success Check

Week 2 worked if you can trace the path.

For a small program, you should be able to explain:

- ▶ what value starts the logic
- ▶ what condition is checked
- ▶ what branch runs
- ▶ what repeats
- ▶ what changes
- ▶ what stops the loop

If you can explain that, the program is small enough and clear enough.

34

Southwest Tech

Traceability Checklist

Follow the path from value to output.

- Starting Value**
Set the initial value.
Example: `count = 1` ✓
- Condition**
Check the condition.
Example: `count <= 5?` ✓
- Branch**
Choose the branch.
Example: Yes (true) ✓
- Repeat**
Do the loop body.
Example: `Add count` ✓
- Change**
Update the value.
Example: `count = count + 1` ✓
- Stop**
Condition becomes false.
Example: `count > 5` ✓
- Output**
Show the final result.
Example: `Total = 15` ✓

A clear path. Every time. Understand. Test. Trust.

Closing Success Check

35

Southwest Tech



Thank you!

Have Fun!

36